

As a final bit of terminology, we say that a non-strict function in Scala takes its arguments *by name* rather than *by value*.

5.2 An extended example: lazy lists

Let's now return to the problem posed at the beginning of this chapter. We'll explore how laziness can be used to improve the efficiency and modularity of functional programs using *lazy lists*, or *streams*, as an example. We'll see how chains of transformations on streams are fused into a single pass through the use of laziness. Here's a simple Stream definition. There are a few new things here we'll discuss next.

Listing 5.2 Simple definition for Stream

A smart constructor for creating a nonempty stream.

A smart constructor for creating an empty stream of a particular type.

```
sealed trait Stream[+A]
case object Empty extends Stream[Nothing]
case class Cons[+A](h: () => A, t: () => Stream[A]) extends Stream[A]

object Stream {
  def cons[A](hd: => A, tl: => Stream[A]): Stream[A] = {
    lazy val head = hd
    lazy val tail = tl
    Cons(() => head, () => tail)
  }
  def empty[A]: Stream[A] = Empty

  def apply[A](as: A*): Stream[A] =
    if (as.isEmpty) empty else cons(as.head, apply(as.tail: _*))
}
```

A nonempty stream consists of a head and a tail, which are both non-strict. Due to technical limitations, these are thunks that must be explicitly forced, rather than by-name parameters.

We cache the head and tail as lazy values to avoid repeated evaluation.

A convenient variable-argument method for constructing a Stream from multiple elements.

Explicit forcing of the h thunk using h()

This type looks identical to our List type, except that the Cons data constructor takes *explicit* thunks `() => A` and `() => Stream[A]` instead of regular strict values. If we wish to examine or traverse the Stream, we need to force these thunks as we did earlier in our definition of `if2`. For instance, here's a function to optionally extract the head of a Stream:

```
def headOption: Option[A] = this match {
  case Empty => None
  case Cons(h, t) => Some(h())
}
```

Note that we have to force `h` explicitly via `h()`, but other than that, the code works the same way as it would for List. But this ability of Stream to evaluate only the portion actually demanded (we don't evaluate the tail of the Cons) is useful, as we'll see.